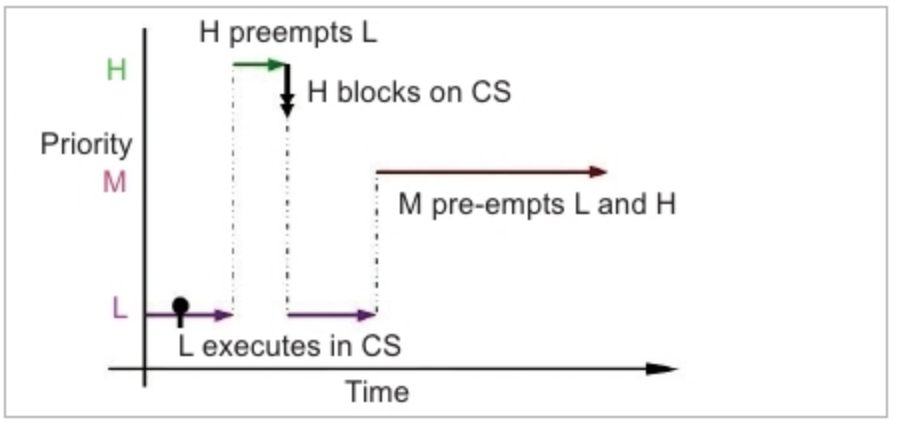

Figure 1: Priority inversion

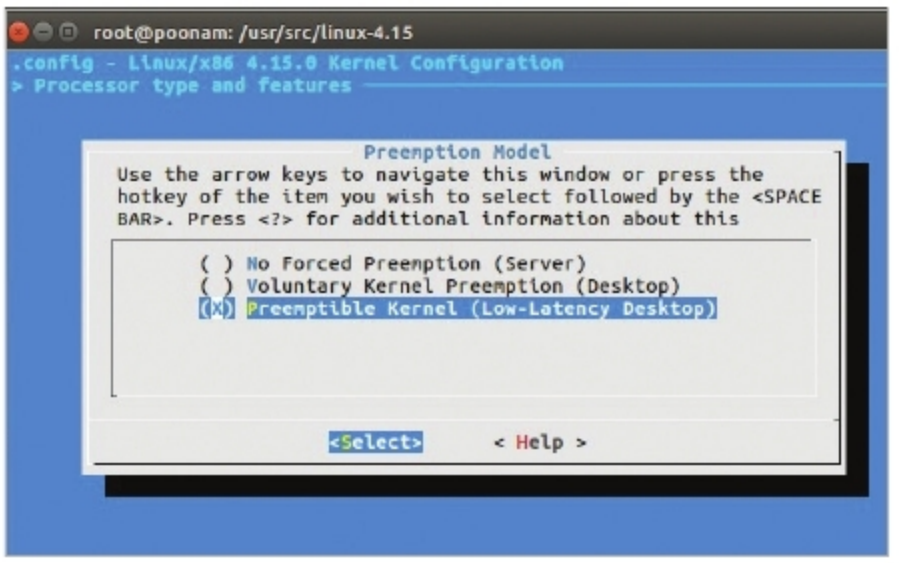

Figure 2: Kernel pre-emption model

Shared resource: Critical section

The shared buffer memory between H and L is protected by a lock (say semaphore). Let's consider the following different scenarios:

- L is running but not in the critical section (CS) and H needs to run. So, in priority based pre-emptive scheduling, H pre-empts L. L then runs once H completes its execution.
- L is running in CS, H needs to run but not necessarily in CS. Even then, H pre-empts L, and then executes. L resumes execution once H finishes its execution.
- L is running in CS. H also needs to run in CS. H waits for L to come out of CS and then executes.

The above three scenarios are normal and don't lead to any problems. The real problem arises when another task of middle priority needs to be executed.

Let us consider a scenario again:

1. L runs, and H tries to access CS while L is executing it.
2. As CS is not available, H sleeps.
3. M arrives. As M has higher priority than L and it doesn't need to be executed in CS, L gets pre-empted by M, which then runs.
4. M finishes, then L executes for the remaining portion and releases CS; only then does H get resources.
5. H executes.

What if M is a CPU bound process which needs a lot of CPU time? In this case, M keeps on executing and apparently, L is waiting for M to get executed, and indirectly H too is waiting on M.

This is known as priority inversion.

An experimental setup

This experimental work is done on Ubuntu 16.04 with kernel version 4.15, and a core processor with 8GB RAM. The vanilla kernel offers the following three options for the pre-emptible kernel. We have used a pre-emptible (low-latency desktop) model as shown in Figure 2.

The source code is available at *https://github.com/Poonam0602/Linux-System-programming/tree/master/Linux%20Kernel%20Locking%20Mechanisms.*

withoutLock.c: In the source code, Lines 1 to 9 represent header files. The *my_read* function is used to retrieve data from the device. A non-negative return value represents the number of bytes successfully read (the return value is a 'signed size' type, usually the native integer type for the target platform). Four arguments should be passed in the read function—the first and second are pointers to the file structure and user buffer, respectively. The third is the size of the data to be read, while the fourth is the file pointer position. When an application program issues the read system call, this function is invoked, and the data is to be copied from the kernel space to the user space using the *copy_to_user* function. *copy_to_user* accepts three arguments—a pointer to the user buffer, a pointer to the kernel buffer and the size of the data to be transferred.

The function *my_write* is similar to *read*. Here, data is transferred from the user space to the kernel space. *Copy_from_user* is used for this data transfer. The delay mentioned in the above function at Line 25 is used to demonstrate how the code works, with screenshots. This delay provides enough time to capture the behaviour on the screen.

Opening the device is the first operation performed on the device file. Two arguments are passed in the *my_open* function. The first argument is the inode structure, which is used to retrieve information such as the user ID, group ID, access time, size, the major and minor number of a device, etc.

The *file_operations* structure (Lines 44 to 49) is an array of function pointers, which is used by the kernel to access the driver's functions. All the device drivers record the *read, write, open* and *release* functions in the *file_operations* structures. The functions *my_read, my_write, my_open* and *my_close* are assigned to the corresponding device entry points.

My_init() function at Lines 53 to 60 is used to register a character device called *mydevice* using *register_chrdev(),* which returns the major number. The zero passed as the parameter enables the dynamic allocation of a major number to the device.

init_module() replaces one of the kernel functions with the *my_init* function. The *exit_module()* function is supposed to undo whatever *init_module()* did, so the module can be unloaded safely.

semaphore.c: This complete code is the same as that of *withoutLock.c* except for some changes mentioned below.

Line 11 represents *DEFINE_Semaphore*, which defines and initialises a global semaphore named *test_Semaphore.*